CRON Jobs - Complete Guide (Hinglish)

Table of Contents

1. CRON Kya Hai?
2. CRON Expression Kaise Likhte Hain?
3. UTC Time Aur Timezone Ka Concept
4. Aapke Codebase Mein CRON Jobs
5. Practical Examples
6. Common Mistakes Aur Solutions

CRON Kya Hai?

CRON ek time-based job scheduler hai jo Unix-like operating systems mein use hota hai. Simple words mein, yeh aapko **scheduled tasks** ko automatically specific time par run karne ki facility deta hai.

Real Life Example:

Imagine karo aapko:

- **Har raat 2 AM** ko database backup lena hai
- **Har Monday subah 9 AM** ko weekly report generate karni hai
- **Har 5 minute** mein koi monitoring task run karni hai

Iske liye aapko manually task run karne ki zaroorat nahi hai. CRON automatically specified time par task execute kar dega.

CRON Jobs Ki Key Features:

1. ✅ **Automated Execution** - Manual intervention ki zaroorat nahi
2. ✅ **Precise Scheduling** - Exact time par run hota hai
3. ✅ **Recurring Tasks** - Baar baar same task repeat kar sakta hai
4. ✅ **Background Process** - Silently background mein run hota hai

---

CRON Expression Kaise Likhte Hain?

CRON expression ek special format string hai jo batata hai ki task kab run hona chahiye.

Standard CRON Format (5 fields):

```
* * * * *
| | | | |
| | | | └──── Day of Week (0-6) (Sunday=0)
| | | └────── Month (1-12)
| | └──────── Day of Month (1-31)
```

```
  │ └──────────── Hour (0–23)
  └────────────── Minute (0–59)
```

Extended CRON Format (6 fields):

```
* * * * * *
| | | | | |
| | | | | └── Year (optional)
| | | | └──── Day of Week (0–6)
| | | └────── Month (1–12)
| | └──────── Day of Month (1–31)
| └────────── Hour (0–23)
└──────────── Minute (0–59)
```

Special Characters:

Character	Meaning	Example
`*`	Har value (Every)	`* * * * *` = Har minute
`,`	Multiple values	`0,15,30,45 * * * *` = 0, 15, 30, 45 minute par
`-`	Range of values	`0-5 * * * *` = 0 se 5 minute tak
`/`	Step values	`*/5 * * * *` = Har 5 minute mein

Common CRON Expression Examples:

Expression	Meaning	Kab Run Hoga
`* * * * *`	Har minute	Har minute
`*/5 * * * *`	Har 5 minute	0, 5, 10, 15, 20... minutes par
`0 * * * *`	Har hour	Har hour ke 0th minute par
`0 0 * * *`	Daily midnight	Har raat 12 AM (00:00)
`0 9 * * *`	Daily 9 AM	Har din subah 9 AM
`0 9 * * 1`	Every Monday 9 AM	Har Monday subah 9 AM
`0 0 1 * *`	Monthly 1st day	Har mahine ki pehli tareek ko midnight
`0 0 1 1 *`	Yearly Jan 1st	Har saal 1st January ko midnight
`*/10 9-17 * * 1-5`	Every 10 min, 9-5 PM, Weekdays	Office hours mein har 10 min

---

UTC Time Aur Timezone Ka Concept

Yeh sabse important aur confusing part hai. Isko detail mein samajhte hain.

UTC Kya Hai?

UTC (Coordinated Universal Time) ek global time standard hai jo **Greenwich Mean Time (GMT)** par based hai. Yeh world ka reference time hai.

Key Points:

- 🌍 **Universal Standard** - Puri duniya mein same
- ⏰ **No Daylight Saving** - Kabhi change nahi hota
- 🎯 **Reference Point** - Sabhi timezones UTC se calculate hote hain

Timezone Offsets:

Location	Timezone	UTC Offset	Example
London, UK	GMT	UTC+0	12:00 PM UTC = 12:00 PM London
Singapore	SGT	UTC+8	12:00 PM UTC = 8:00 PM Singapore
India	IST	UTC+5:30	12:00 PM UTC = 5:30 PM India
New York	EST	UTC-5	12:00 PM UTC = 7:00 AM New York
Los Angeles	PST	UTC-8	12:00 PM UTC = 4:00 AM LA

Server Time UTC Mein Kyu Hota Hai?

1. Consistency (Ekroop-ta)

```
Problem: Agar server India mein hai aur IST use karta hai
- Ek user India mein hai (IST timezone)
- Doosra user Singapore mein hai (SGT timezone)
- Teesra user USA mein hai (EST timezone)

Confusion: Sabke liye alag time interpretation hoga!
```

```
Solution: Server UTC use karta hai
- Server: "Task run at 12:00 UTC"
- India user: Apne local time mein convert karke dekhega (5:30 PM IST)
- Singapore user: Apne local time mein convert karke dekhega (8:00 PM SGT)
- USA user: Apne local time mein convert karke dekhega (7:00 AM EST)

Sab consistent rahega!
```

2. No Daylight Saving Confusion

Kuch countries mein Daylight Saving Time (DST) hota hai jisme clock aage-piche hota hai. UTC kabhi change nahi hota, isliye confusion nahi hoti.

3. Global Coordination

Distributed systems mein jab multiple servers different locations par hain, tab UTC ek common ground provide karta hai.

4. Database Consistency

Database mein timestamp hamesha UTC mein store karte hain, phir application layer par user ke timezone mein convert karte hain.

Real Example Scenario:

```
Maan lo aapko ek CRON job set karna hai:
"Har subah 9 AM Singapore time par report generate karo"

Option 1: Server UTC mein hai (Recommended ✅)
-------------------------
Singapore time: 9:00 AM SGT
UTC equivalent: 1:00 AM UTC (kyunki SGT = UTC+8)

CRON expression: 0 1 * * *
CRON scheduler timezone: UTC

Yeh hamesha 9 AM SGT par hi run hoga, chahe server kahi bhi ho!


Option 2: Server local timezone use kare (Not recommended ❌)
-------------------------
Agar server move ho jaye different region mein, toh:
- Confusion hogi
- Time calculations galat ho jayenge
- CRON expression change karni padegi
```

Aapke Codebase Mein CRON Jobs

Aapke codebase mein **2 different CRON libraries** use ho rahi hain:

1. **Batchify Service** (`github.com/go-co-op/gocron`)

File: `batchify/generic/controller/cron_controller.go`

```
// Scheduler initialize karte waqt
scheduler: gocron.NewScheduler(time.Now().Location())
```

Yahan par kya ho raha hai?

- `time.Now().Location()` current system ka timezone use karta hai

- Agar server UTC mein hai, toh scheduler bhi UTC use karega
- Agar aapke config mein timezone set hai, toh woh use hoga

CRON Expression Config: `batchify/config_files/service-conf.json`

```
{
  "timeZone": "Asia/Singapore",  // ⬅️ Application ka base timezone
  "assetCampaignWorkerConfig": {
    "cronExpression": [
      "*/1 * * * *"  // ⬅️ Har 1 minute mein run hoga
    ]
  },
  "jobConfigs": [
    {
      "name": "invest_recon",
      "isEnabled": true,
      "sourceType": "sftp-cron",
      "sourceProperties": {
        "cronExpression": [
          "*/1 * * * *"  // ⬅️ Har 1 minute mein run hoga
        ]
      }
    }
  ]
}
```

Important: Yahan `timeZone: "Asia/Singapore"` set hai, matlab:

- Agar CRON expression `0 9 * * *` ho
- Toh yeh **Singapore time ke 9 AM** par run hoga
- Server internally isko UTC mein convert kar lega (1 AM UTC)

---

2. **Customer-Experience Service** (`github.com/robfig/cron/v3`)

File: `customer-experience/scheduler/scheduler.go`

```
func (c *Cron) Init(ctx context.Context, appCfg *config.AppConfig) {
    // Config se timezone load karo
    location, err := time.LoadLocation(appCfg.CronJobsConfig.TimeZone)
    if err != nil {
        panic(err)
    }

    // CRON client ko specified timezone ke saath initialize karo
    c.CronClient = cron.New(cron.WithLocation(location))

    // CRON job add karo
    c.CronClient.AddFunc(businessMonitoringEmailCfg.Schedule,
```

```
c.BusinessMonitoringEmailCron.Run)

    // CRON start karo
    c.CronClient.Start()
}
```

CRON Config: `customer-experience/config_files/service-conf.json`

```
{
  "cronJobsConfig": {
    "isEnabled": true,
    "timeZone": "Asia/Singapore",  // ⬅️ Timezone explicitly set hai
    "monitoringCronConfig": {
      "BWS": {
        "isEnabled": true,
        "schedule": "*/5 * * * *",  // ⬅️ Har 5 minute mein run hoga
        "lockDuration": 60
      }
    }
  }
}
```

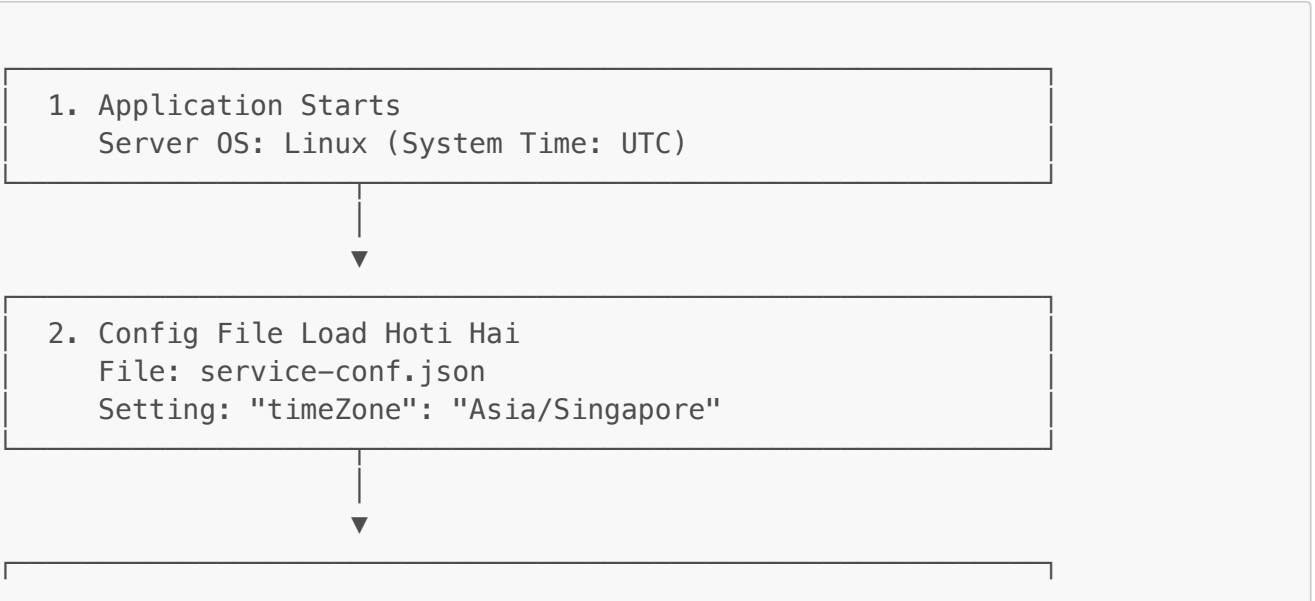
Yahan kya ho raha hai?

1. Config mein `timeZone: "Asia/Singapore"` set hai
2. Code `time.LoadLocation()` use karke Singapore timezone load karta hai
3. CRON client ko explicitly yeh timezone assign kiya jata hai
4. Ab CRON expressions Singapore time ke according interpret honge

---

Timezone Selection Process - Step by Step

Process Flow:



```
┌─────────────────────────────────────────────────────────────┐
│  3. Scheduler Initialize Hota Hai                           │
│     Code: time.LoadLocation("Asia/Singapore")               │
│     Result: Singapore timezone object mil gaya              │
└─────────────────────────────────────────────────────────────┘
                          │
                          ▼
┌─────────────────────────────────────────────────────────────┐
│  4. CRON Client Create Hota Hai                             │
│     Code: cron.New(cron.WithLocation(locationSGT))          │
│     CRON: Ab Singapore timezone use karega                  │
└─────────────────────────────────────────────────────────────┘
                          │
                          ▼
┌─────────────────────────────────────────────────────────────┐
│  5. CRON Expression Parse Hota Hai                          │
│     Expression: "0 9 * * *"                                 │
│     Interpretation: 9 AM Singapore Time                     │
└─────────────────────────────────────────────────────────────┘
                          │
                          ▼
┌─────────────────────────────────────────────────────────────┐
│  6. Internal Conversion (UTC mein)                          │
│     Singapore 9 AM = UTC 1 AM (because SGT = UTC+8)         │
│     CRON library internally UTC mein track karta hai        │
└─────────────────────────────────────────────────────────────┘
                          │
                          ▼
┌─────────────────────────────────────────────────────────────┐
│  7. Execution Time                                          │
│     Server Time (UTC): 01:00                                │
│     Singapore Time: 09:00                                   │
│     ✅ CRON Job Execute Hota Hai                            │
└─────────────────────────────────────────────────────────────┘
```

Database Connection Mein Bhi UTC:

Dekho aapke config mein:

```
{
  "mysql": {
    "master": {
      "dsn": "root:@tcp($MYSQL_HOST$:3306)/batchify?parseTime=true&loc=UTC"

^^^ UTC set hai
    }
  }
}
```

Yeh ensure karta hai ki database mein bhi sab timestamps UTC mein store hoon.

Practical Examples

Example 1: Daily Report - 9 AM Singapore Time

```
{
  "timeZone": "Asia/Singapore",
  "cronJobsConfig": {
    "dailyReport": {
      "schedule": "0 9 * * *",  // 9 AM SGT har roz
      "isEnabled": true
    }
  }
}
```

Kya Hoga:

- Singapore mein: 9:00 AM par run hoga
- UTC mein: 1:00 AM par run hoga (internally)
- India mein agar koi dekhe: 6:30 AM IST par notification milega

---

Example 2: Every 5 Minutes - Business Monitoring

```
{
  "timeZone": "Asia/Singapore",
  "monitoringCronConfig": {
    "BWS": {
      "schedule": "*/5 * * * *",  // Har 5 minute mein
      "isEnabled": true
    }
  }
}
```

Kya Hoga:

- Har 5 minute mein run hoga: 00, 05, 10, 15, 20...
- Timezone ka yahan zyada farak nahi padta kyunki interval-based hai
- Lekin agar koi time-based filtering ho (9 AM - 5 PM), toh Singapore time ke according hoga

---

Example 3: Weekly Report - Monday 10 AM

```
{
  "timeZone": "Asia/Singapore",
  "cronJobsConfig": {
    "weeklyReport": {
      "schedule": "0 10 * * 1",  // Har Monday 10 AM SGT
```

```
      "isEnabled": true
    }
  }
}
```

Kya Hoga:

- Har Monday Singapore time ke 10 AM par
- Agar Monday Singapore mein start ho chuka ho lekin UTC mein Sunday hai, koi problem nahi
- CRON scheduler Singapore time ko track karega

Example 4: Month End Processing - Last Day 11:59 PM

```
{
  "timeZone": "Asia/Singapore",
  "cronJobsConfig": {
    "monthEndProcess": {
      "schedule": "59 23 28-31 * *",  // Last days of month 11:59 PM
      "isEnabled": true,
      "checkLastDay": true  // Extra logic to verify it's actually last day
    }
  }
}
```

Kya Hoga:

- Month ke last 4 days (28-31) ko check karega
- Application logic se verify karega ki actual last day hai ki nahi
- 11:59 PM Singapore time par run hoga

Common Mistakes Aur Solutions

❌ Mistake 1: Timezone Ignore Karna

```
// WRONG ❌
scheduler := gocron.NewScheduler(time.UTC)
// Har expression UTC ke according interpret hoga

// Config mein likha: "0 9 * * *" (9 AM expectation)
// Actual run time: 9 AM UTC = 5 PM SGT (Galat!)
```

```
// CORRECT ✅
location, _ := time.LoadLocation("Asia/Singapore")
scheduler := gocron.NewScheduler(location)
```

```
// Har expression Singapore time ke according interpret hoga

// Config mein likha: "0 9 * * *"
// Actual run time: 9 AM SGT = 1 AM UTC (Sahi!)
```

❌ Mistake 2: Server Time Assume Karna

```
// WRONG ❌
// Developer sochta hai: "Server India mein hai, toh IST use hoga"
scheduler := gocron.NewScheduler(time.Now().Location())

// Actual: Server UTC mein hai
// Result: Sab times galat ho jayenge!
```

```
// CORRECT ✅
// Explicitly timezone config se load karo
location, _ := time.LoadLocation(config.TimeZone)
scheduler := gocron.NewScheduler(location)

// Ab chahe server kahi bhi ho, timezone consistent rahega
```

❌ Mistake 3: Daylight Saving Time Confusion

```
// WRONG ❌ (US/Eastern DST ke wajah se)
location, _ := time.LoadLocation("America/New_York")
scheduler := gocron.NewScheduler(location)

// Summer mein: EDT (UTC-4)
// Winter mein: EST (UTC-5)
// CRON timing change ho jayegi!
```

```
// CORRECT ✅
// DST-free timezone use karo ya UTC use karo
location, _ := time.LoadLocation("UTC")
scheduler := gocron.NewScheduler(location)

// Ya phir fixed offset use karo
location := time.FixedZone("EST", -5*3600)
```

❌ Mistake 4: Database Aur Application Timezone Mismatch

```
-- database.sql
-- Timestamp without timezone information
CREATE TABLE jobs (
    executed_at TIMESTAMP  -- ⚠️ Timezone info nahi hai
);
```

```
// Application code
executedAt := time.Now()  // Local timezone use ho raha hai
db.Exec("INSERT INTO jobs (executed_at) VALUES (?)", executedAt)

// Problem: Database UTC expect kar raha, app local time send kar raha
```

Solution:

```
-- Use TIMESTAMP with explicit UTC
CREATE TABLE jobs (
    executed_at TIMESTAMP  -- Store in UTC
);
```

```
// Always convert to UTC before saving
executedAt := time.Now().UTC()
db.Exec("INSERT INTO jobs (executed_at) VALUES (?)", executedAt)

// When reading, convert to user's timezone
executedAt := row.ExecutedAt
localTime := executedAt.In(userLocation)
```

Testing CRON Jobs

Local Development Mein Test Kaise Kare?

Method 1: Short Interval Use Karo

```
{
  "cronJobsConfig": {
    "testJob": {
      "schedule": "*/1 * * * *",  // Har minute (testing ke liye)
      "isEnabled": true
    }
  }
}
```

Method 2: Manual Trigger Function Banao

```
// Add a test endpoint
func TestCronJob(w http.ResponseWriter, r *http.Request) {
    ctx := r.Context()
    cronHandler.Run()  // Manually trigger
    w.WriteHeader(http.StatusOK)
}
```

Method 3: Time Mock Karo (Unit Tests)

```
// In test
func TestCronSchedule(t *testing.T) {
    // Mock time
    now := time.Date(2024, 1, 1, 9, 0, 0, 0, time.UTC)

    // Test if CRON should trigger
    shouldTrigger := cronExpression.Next(now)

    assert.Equal(t, expectedTime, shouldTrigger)
}
```

Quick Reference Card

CRON Expression Cheat Sheet

```
# ┌──────── minute (0 - 59)
# │ ┌────── hour (0 - 23)
# │ │ ┌──── day of month (1 - 31)
# │ │ │ ┌── month (1 - 12)
# │ │ │ │ ┌ day of week (0 - 6) (Sunday=0)
# │ │ │ │ │
# * * * * *

# Common Patterns:
*/5 * * * *      # Har 5 minute
0 */2 * * *      # Har 2 hours
0 9-17 * * 1-5   # Office hours (9 AM - 5 PM, Mon-Fri)
0 0 * * *        # Daily midnight
0 0 * * 0        # Weekly (Sunday midnight)
0 0 1 * *        # Monthly (1st day midnight)
0 0 1 1 *        # Yearly (January 1st)
```

Timezone Conversion Table

SGT (UTC+8)	UTC	IST (UTC+5:30)	EST (UTC-5)
12:00 AM	4:00 PM (prev day)	9:30 PM (prev day)	11:00 AM (prev day)
9:00 AM	1:00 AM	6:30 AM	8:00 PM (prev day)
12:00 PM	4:00 AM	9:30 AM	11:00 PM (prev day)
6:00 PM	10:00 AM	3:30 PM	5:00 AM
11:59 PM	3:59 PM	9:29 PM	10:59 AM

Advanced Topics

1. CRON Job Locking (Distributed Systems)

Agar multiple server instances hain, toh ek hi CRON job multiple times run ho sakta hai. Isko prevent karne ke liye:

```
// Redis lock use karo
func (c *CronHandler) Run() {
    lockKey := "cron:job:business_monitoring"

    // Try to acquire lock
    acquired, err := redis.SetNX(lockKey, "locked",
60*time.Second).Result()
    if !acquired {
        log.Info("Job already running on another instance")
        return
    }
    defer redis.Del(lockKey)

    // Execute actual job
    c.executeBusinessMonitoring()
}
```

Yeh aapke config mein bhi hai:

```
{
  "monitoringCronConfig": {
    "BWS": {
      "lockDuration": 60  // ⬅️ 60 seconds ka lock
    }
  }
}
```

---

2. CRON Job Monitoring

```go
// Job execution track karo
func (c *CronHandler) Run() {
    startTime := time.Now()

    defer func() {
        duration := time.Since(startTime)
        metrics.RecordCronDuration("business_monitoring", duration)

        if r := recover(); r != nil {
            metrics.RecordCronFailure("business_monitoring")
            log.Error("CRON job panicked", slog.Any("error", r))
        }
    }()

    // Actual job execution
    err := c.executeBusinessMonitoring()
    if err != nil {
        metrics.RecordCronFailure("business_monitoring")
        return
    }

    metrics.RecordCronSuccess("business_monitoring")
}
```

---

3. Graceful Shutdown

```go
func (s *Server) Shutdown(ctx context.Context) error {
    log.Info("Stopping CRON scheduler")

    // CRON scheduler ko gracefully stop karo
    s.cronScheduler.Stop()

    // Running jobs ko complete hone do
    select {
    case <-ctx.Done():
        log.Warn("Shutdown timeout, forcing exit")
        return ctx.Err()
    case <-s.waitForJobs():
        log.Info("All CRON jobs completed")
        return nil
    }
}
```

Summary

Key Takeaways:

1. ✅ **CRON = Automated Scheduler**
   - Time-based tasks automatically execute karte hain
   - Manual intervention ki zaroorat nahi
2. ✅ **UTC = Global Standard**
   - Server time hamesha UTC mein hona chahiye
   - Consistency aur coordination ke liye important
3. ✅ **Timezone Configuration**
   - Application level par timezone set karo
   - CRON expressions ko us timezone ke according interpret karo
   - Internally UTC mein convert ho jaye
4. ✅ **Testing Important Hai**
   - Short intervals use karo local testing ke liye
   - Manual triggers banao debugging ke liye
   - Time mocking karo unit tests mein
5. ✅ **Production Considerations**
   - Distributed locking use karo
   - Monitoring aur alerting setup karo
   - Graceful shutdown implement karo

Useful Commands

Check Server Timezone

```
# Current timezone dekho
timedatectl

# System time check karo
date

# UTC time check karo
date -u
```

Test CRON Expression Online

```
https://crontab.guru/
```

Debug CRON Jobs

```
# Application logs dekho
tail -f /var/log/application.log | grep "cron"

# Specific time par kya hua?
grep "2024-06-05 09:00" /var/log/application.log
```

Resources

1. **CRON Expression Tester**: https://crontab.guru/
2. **Timezone Converter**: https://www.timeanddate.com/worldclock/converter.html
3. **Go Time Package**: https://pkg.go.dev/time
4. **gocron Library**: https://github.com/go-co-op/gocron
5. **robfig/cron**: https://github.com/robfig/cron

Happy Scheduling! 🚀

Agar koi doubt ho toh Slack ya Email par poocho. CRON jobs se daro mat, yeh bahut powerful tool hai! ✨